

Recod.ai/LUC - Scientific Image Forgery Detection

Max Bakke Seymour¹, Einar Martin Søvik Bernsen¹, and Emre Balci¹

¹University of Bergen (UiB)

Abstract

TODO - "Short summary of the paper, including results. Often written last". We additionally implemented a SegNeXt-inspired segmentation baseline and compared it with the project's DINO-based baseline. While the DINO-based model achieved the best validation F1 score of 0.1850, the best pretrained SegNeXt-family variant reached a comparable score of 0.1827, showing that learned segmentation approaches can be competitive on this task.

1 Introduction

1.1 Copy-Move Forgery Detection

Copy-Move Forgery is a class of image forgery where regions of an image are duplicated and moved to other areas of the image. Commonly, techniques such as rotation, flipping and scaling of the copied region are applied, as well as post-processing such as edge smoothing, brightness adjustment and adding extra noise. These techniques can lead to forgeries which are hard to detect both through manual review and automated detectors. This method of forgery can be used within scientific images in order to fabricate results, where the complexity of the figures creates additional difficulties. [kaggle]

1.2 Related works

Copy-move image forgery detection has been studied through the lens of many different techniques. Early approaches focused on hand-crafted descriptors extracted either densely over overlapping blocks or sparsely at keypoints, followed by nearest-neighbour matching and geometric consistency checks [fridrich2003copymove, christlein2012evaluation]. These methods established the basic copy-move detection pipeline and showed that explicit correspondence search can be effective, but they are often sensitive to strong post-processing, complex geometric transformations, and repeated structures in the scene [christlein2012evaluation].

In more recent work, the focus has shifted towards deep learning for copy-move detection. Some methods retain the classical matching intuition while replacing hand-crafted features

with learned representations and stronger dense matching modules [liu2022twostage]. Others formulate the task as end-to-end forgery localization, using CNN-based architectures to jointly detect duplicated content and localize manipulated regions [wu2018busternet]. Attention-based models have further improved localization by explicitly modelling patch relationships and co-occurrence cues [islam2020doagan], while more recent transformer-based approaches have explored global context modelling for copy-move detection [liu2024cmfdformer]. In parallel, source-target disambiguation has also emerged as an important subproblem, since detecting near-duplicate regions alone is not always sufficient to identify the truly manipulated area [barni2021copymove].

One of the other architectures we tried for this task even combines both perspectives by using CNN crafted features in combination with features computed through Zernike kernels. [li2024deepmatch]

1.3 Objectives

We aim to create a full prediction pipeline for CMFD which could perform well on the Recod.AI Scientific Image Forgery Detection dataset. Ideally, our model should be robust towards post-processing techniques. The main goal is to perform well on the competition metric, which is a modified version of F1 which computes pixelwise F1 on pairs of (*predicted mask*, *ground truth mask*) using the following formula [metric]:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

This calculation is applied to every connected component of forged pixels in the image (belonging to either the source or target). Predictions given by the model are matched up with the best corresponding ground-truth areas using the Hungarian algorithm. For any additional predicted areas predicted by the model above the actual amount of ground-truth components, a penalty is computed:

$$P = \frac{n_{gt}}{\max(n_{pred}, n_{gt})}$$

Finally, the per-image score is given as:

$$oF1 = \text{mean}(\text{best matched pairwise F1 scores}) \times P$$

The competition metric is challenging for a few reasons. Firstly, it requires high pixel-level precision for the output masks. In addition, a score of 0 for a given image is given if any pixels are predicted as forged on a ground-truth authentic image. Finally, it is highly punishing if any additional components are predicted, even if they should be small, as this affects the penalty P .

1.4 Contributions

This paper presents a complete main pipeline for CMFD and several alternate methods, discussing the strengths and weaknesses of the different approaches. Through a discussion and presentation of these methods, we aim to identify what directions could be promising for future work and further nuance the task of CMFD.

2 Methods

Our initial implementation was inspired by a notebook by Kaggle user Gaurav Parkhedkar [notebook], who successfully employed a range of different techniques to perform a score of 0.332 on the competition metric on the test set. These techniques include feature extraction through the DINOv2 model [oquab2023dinov2], sliding window inference and prediction mask post-processing. For our own implementation we build on these techniques to try other possibilities.

2.1 Dataset

We utilize the dataset supplied in the Kaggle competition [kaggle]. Unfortunately, we were unable to test our implementation against the Kaggle test-set, as we were too late to enter the competition. As we did not have access to the test data, we instead chose to use the available training data in a train (80%), evaluate (10%), test (10%) split. The training data consists of a base 2751 forged and 2377 authentic images as well as an additional 48 forged images. As the images are too large to process directly with the model, they are each resized to size $3 \times 448 \times 448$.

2.2 Feature Extraction

To perform feature extraction on the images, we employ Meta AI’s powerful DINOv2 model using the pretrained *ViT-B/14* segmentation head [oquab2023dinov2]. This model first extracts strong general-purpose features using the DINOv2 ViT backbone, then uses these features to create pixel-level predictions using the segmentation head.

This model is kept frozen and applied on 14×14 size patches of the image at a time to generate features with an embedding dimension of 768. When predicting,

2.3 Decoder

In order to create the predicted pixel mask, we employ a small decoder model on the generated DINOv2 feature map. The decoder we utilize contains three blocks and an output layer, before outputting the generated mask interpolated to output size using bilinear interpolation.

The first two blocks of the decoder apply convolutions to bring the dimensionality from $768 \rightarrow 384 \rightarrow 192$ while also applying *ReLU* and *Dropout* with $p = 0.1$ operations in each block. In the third block, a convolution from $192 \rightarrow 96$ is applied before a final *ReLU* and the output layer $96 \rightarrow 1$, giving the predicted mask.

2.4 Mask post-processing

To post-process masks, we applied the following operations to an input map of forged pixel probabilities:

1. Gaussian blur: A smoothing operation implemented through `scipy.ndimage.gaussian_filter(mask, sigma=0.5)`
2. Confidence threshold filtering: Creates an initial binary mask by filtering probabilities on the threshold 0.5.
3. Restore very confident pixels: Restores pixels predicted with a confidence ≥ 0.9 , as they may have been lost through smoothing, and combines them with the mask generated through confidence threshold filtering.
4. Closing operation: Applies dilation followed by erosion in order to create more coherent objects, implemented through `scipy.ndimage.binary_closing(mask, structure=np.ones((5, 5)))`
5. Opening operation: Applies erosion followed by dilation in order to remove small isolated blobs, implemented through `scipy.ndimage.binary_opening(mask, structure=np.ones((3, 3)))`
6. Finally, filling any remaining holes in binary objects using `scipy.ndimage.binary_fill_holes(mask)`

2.5 Sliding window inference

In order to learn global relationships between pixels in an image, it is necessary to run inference on images as a whole. This is performed on images of the same resized size as the training image input size. The generated pixel masks are upsampled through bilinear interpolation, similarly to the training image samples.

The issue with this kind of inference is that although it is able to learn the global relationships between objects in the image, it is challenging to achieve perfect pixel-level precision on images when the resolution is decreased significantly. In order to give our model more pixel-level precision and allow for inference on higher-resolution images directly, we employ sliding window inference. Sliding window inference is performed by performing individual predictions on overlapping patches of the image, which are weighted by a gaussian kernel in order to create a final output pixel map. TODO: More detailed: This is combined with the global prediction performed on the whole downsampled image at once.

2.6 SegNeXt-inspired Baseline and Experimental Comparison

In addition to the main DINO-based pipeline, we implemented a learning-based baseline inspired by SegNeXt in order to compare a convolutional segmentation architecture against the project baseline. SegNeXt is motivated by the idea that semantic segmentation can benefit from a strong encoder, multi-scale feature interaction, and efficient spatial attention implemented using convolutional operations rather than transformer self-attention [segnext]. In particular, the original SegNeXt design uses a convolutional encoder-decoder architecture with multi-scale convolutional attention to capture contextual information efficiently. The paper reports strong performance-computation trade-offs and shows that such CNN-based designs can outperform transformer-based alternatives on several semantic segmentation benchmarks [segnext].

Our goal was not to reproduce the full original SegNeXt architecture exactly, but rather to implement a SegNeXt-inspired segmentation baseline within the project pipeline and evaluate whether this type of learned segmentation model could provide competitive performance on the Recod.ai/LUC scientific image forgery detection task. Across these settings, learning rates of 10^{-4} , 5×10^{-5} , 3×10^{-5} , and 10^{-5} were evaluated.

Model	Pretrained	Train Size	Epochs
DINO (baseline)	Yes	900	8
SegNeXt-inspired (custom)	No	500	5
SegNeXt (pretrained encoder)	Yes	900	12

Table 1. Comparison of DINO baseline and SegNeXt-inspired variants.

Experimental setup. The experiments were carried out on the same extracted dataset used in the rest of the project. A subset of the forged training images and corresponding masks was prepared from the Kaggle dataset. For the largest experiments, 1000 image-mask pairs were extracted, of which the effective split used during training corresponded to 900 training samples and 100 validation samples. All experiments were run with the same random seed in order to make comparisons more controlled.

Compared models. We compared three model families:

1. a DINO-based baseline used in the main project,
2. a custom SegNeXt-inspired segmentation model,
3. a pretrained backbone variant using the same lightweight decoder structure.

The DINO-based baseline served as the reference point, while the SegNeXt-inspired experiments were used to test whether a learned convolutional segmentation model could match or improve the baseline under the same training regime.

Training protocol. We trained the models with several dataset sizes and hyperparameter settings. The main experiments used training subsets of size 200, 500, and 1000, with corresponding validation subsets of size 30, 70, and 120. In practice, the largest configuration resulted in an effective split of 900 training images and 100 validation images. We evaluated learning rates of 10^{-4} , 5×10^{-5} , 3×10^{-5} , and 10^{-5} , and trained for between 3 and 12 epochs depending on the experiment.

SegNeXt-inspired implementation. The implemented model followed the general encoder-decoder idea used in modern semantic segmentation architectures. The SegNeXt paper emphasizes multi-scale convolutional feature extraction and efficient spatial attention through convolutional blocks rather than quadratic-complexity self-attention [segnext]. In our implementation, we adopted this design direction as a baseline for scientific forgery localization, while keeping the model lightweight enough to train within the project constraints.